Simple Neural Networks and Neural Language Models

James Pustejovsky

CS 115B — Fundamentals of NLP
Brandeis University

February 6, 2026

Based on Jurafsky & Martin, *Speech and Language Processing*, Ch. 7

Outline

1. Neural Units
2. The XOR Problem
3. Feedforward Neural Networks
4. Feedforward Networks for Classification
5. Embeddings as Input
6. Feedforward Neural Language Models
7. Training Overview
8. Computation Graphs and Backpropagation
9. Putting It All Together
10. Summary and Preview

The Neural Unit: Formal Definition

Definition: Neural Unit

A **neural unit** is a parameterized function $f_{\mathbf{w},b} : \mathbb{R}^n \to \mathbb{R}$ defined by:

1. A **pre-activation** (affine map): $z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{i=1}^{n} w_i x_i + b$
2. A **non-linear activation**: $a = g(z)$

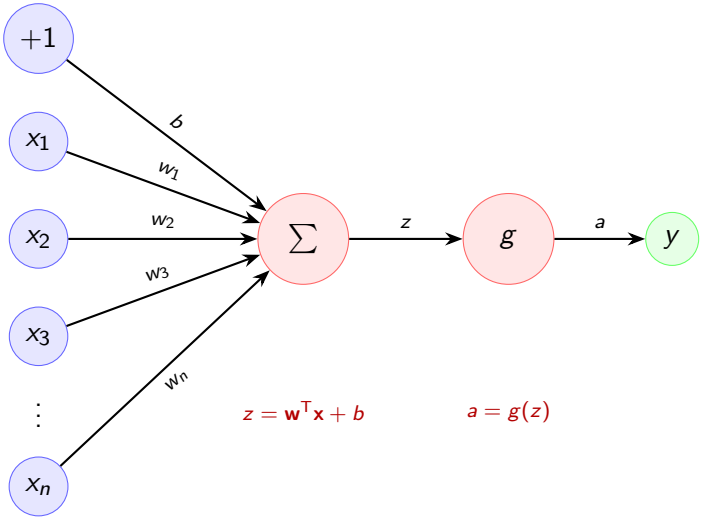
where $\mathbf{w} \in \mathbb{R}^n$ is the **weight vector**, $b \in \mathbb{R}$ is the **bias**, and $g : \mathbb{R} \to \mathbb{R}$ is a non-linear function.

In vector notation:

$$\boxed{y = g(\mathbf{w} \cdot \mathbf{x} + b) = g(\mathbf{w}^\mathsf{T}\mathbf{x} + b)}$$

The output y of the unit depends on three things: the input $\mathbf{x}$, the learnable parameters $(\mathbf{w}, b)$, and the choice of activation g.

The Neural Unit: Diagram



Worked Example: A Single Sigmoid Unit

title

Let $\mathbf{w} = [0.2,\ 0.3,\ 0.9]^{\mathsf{T}}$, $b = 0.5$, $\mathbf{x} = [0.5,\ 0.6,\ 0.1]^{\mathsf{T}}$, $g = \sigma$.

Step 1: Pre-activation.

$$z = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b = (0.2)(0.5) + (0.3)(0.6) + (0.9)(0.1) + 0.5 = 0.10 + 0.18 + 0.09 + 0.5 = 0.87$$

Step 2: Activation.

$$y = \sigma(z) = \frac{1}{1 + e^{-0.87}} = \frac{1}{1 + 0.4190} \approx 0.70$$

Interpretation

For binary classification, the output 0.70 can be read as $P(y = 1 \mid \mathbf{x}) = 0.70$.

The Sigmoid Activation

Sigmoid (Logistic) Function

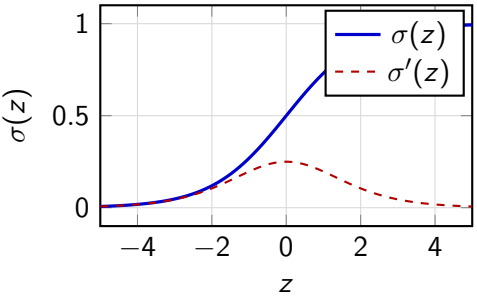
$$\sigma(z) = \frac{1}{1 + e^{-z}} \qquad \sigma : \mathbb{R} \to (0, 1)$$

Properties:

- $\sigma(0) = 0.5, \quad \lim_{z \to \infty} \sigma(z) = 1, \lim_{z \to -\infty} \sigma(z) = 0$
- $1 - \sigma(z) = \sigma(-z)$ (symmetry)
- Derivative:

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

- $\max_z \sigma'(z) = 0.25$ at $z = 0$



Deriving the Sigmoid Derivative

Proposition

If $\sigma(z) = (1 + e^{-z})^{-1}$, then $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

Proof. Let $u = 1 + e^{-z}$, so $\sigma = u^{-1}$. By the chain rule:

$$\sigma'(z) = -u^{-2} \cdot \frac{du}{dz} = -\frac{1}{(1 + e^{-z})^2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Now observe:

$$\sigma(z)\big(1 - \sigma(z)\big) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma'(z) \qquad \square$$

Why This Matters

This self-referential derivative makes gradient computation extremely efficient: given the forward-pass value $\sigma(z)$, the gradient is free.

Other Activation Functions: tanh and ReLU

Hyperbolic Tangent

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

- Range: $(-1, +1)$, zero-centered
- $\tanh(z) = 2\sigma(2z) - 1$
- $\tanh'(z) = 1 - \tanh^2(z)$
- $\max_z \tanh'(z) = 1$ at $z = 0$

Rectified Linear Unit

$$\text{ReLU}(z) = \max(0, z)$$

- Range: $[0, \infty)$
- $\text{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$
- Non-differentiable at $z = 0$; use subgradient $\{0, 1\}$
- **Most common** in modern networks

Why Non-Linearity is Essential

Proposition: Composition of Affine Maps is Affine

If $f_1(\mathbf{x}) = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$ and $f_2(\mathbf{x}) = \mathbf{W}_2\mathbf{x} + \mathbf{b}_2$, then

$$(f_2 \circ f_1)(\mathbf{x}) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{(\mathbf{W}_2\mathbf{W}_1)}_{\widetilde{\mathbf{W}}}\mathbf{x} + \underbrace{(\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)}_{\widetilde{\mathbf{b}}}$$

which is again an affine map $\widetilde{\mathbf{W}}\mathbf{x} + \widetilde{\mathbf{b}}$.

Consequence: Stacking linear layers without non-linear activations g collapses to a single linear layer. The network gains no representational power from depth.
The activation function g **breaks** the linearity, enabling the network to learn non-linear decision boundaries and representations.

Perceptrons: Binary Threshold Units

title

A **perceptron** is a neural unit with the Heaviside step function:

$$y = \begin{cases} 1 & \text{if } \mathbf{w}^\mathsf{T}\mathbf{x} + b > 0 \\ 0 & \text{if } \mathbf{w}^\mathsf{T}\mathbf{x} + b \leq 0 \end{cases}$$

The perceptron implements a **linear classifier**: the decision boundary is the hyperplane

$$\mathcal{H} = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{w}^\mathsf{T}\mathbf{x} + b = 0\}$$

which divides $\mathbb{R}^n$ into two half-spaces.

In $\mathbb{R}^2$ with inputs (x_1, x_2), this is the line:

$$w_1 x_1 + w_2 x_2 + b = 0 \qquad \Longleftrightarrow \qquad x_2 = -\frac{w_1}{w_2}\, x_1 - \frac{b}{w_2}$$

Boolean Functions: AND and OR

AND:

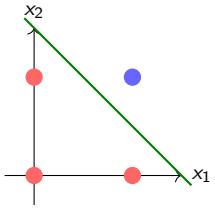
x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

Perceptron: $w_1 = 1,\ w_2 = 1,\ b = -1.5$
Decision boundary: $x_1 + x_2 = 1.5$
Separates $(1, 1)$ from the rest. ✓

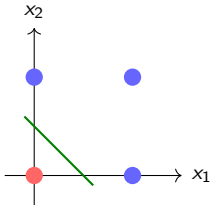
OR:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

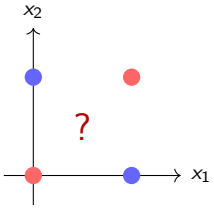
Perceptron: $w_1 = 1,\ w_2 = 1,\ b = -0.5$
Decision boundary: $x_1 + x_2 = 0.5$
Separates $(0, 0)$ from the rest. ✓



AND



OR



XOR

XOR is Not Linearly Separable

XOR: $y = x_1 \oplus x_2$

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Theorem – Minsky and Papert

No single perceptron can compute XOR. Equivalently, the set $\{(0,1),(1,0)\}$ (class 1) and $\{(0,0),(1,1)\}$ (class 0) are **not linearly separable** in $\mathbb{R}^2$.

Proof sketch. Suppose $\exists\, w_1, w_2, b$ such that:

$$b \leq 0 \qquad \text{(input } (0,0) \mapsto 0)$$

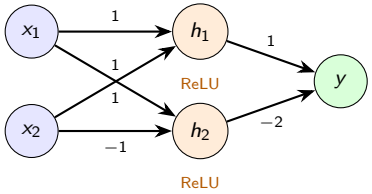
$$w_2 + b > 0 \qquad \text{(input } (0,1) \mapsto 1)$$

Solving XOR with a Two-Layer Network

Architecture: 2 hidden units with ReLU

$$\mathbf{W}^{[1]} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{b}^{[1]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$\mathbf{w}^{[2]} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad b^{[2]} = 0$$



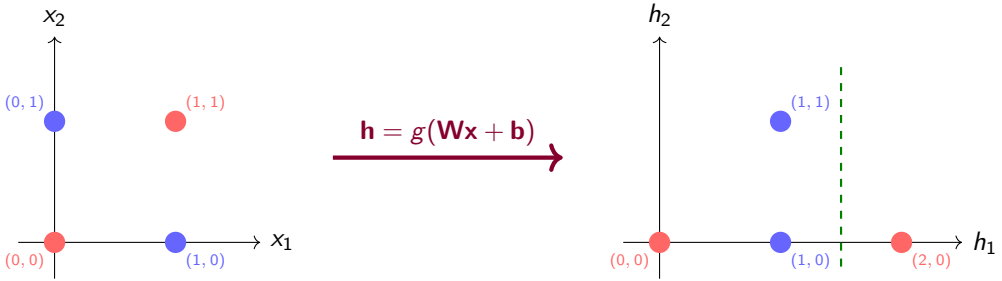
Forward pass for all inputs:

x_1	x_2	h_1	h_2	y
0	0	$\text{ReLU}(0) = 0$	$\text{ReLU}(0) = 0$	0
0	1	$\text{ReLU}(1) = 1$	$\text{ReLU}(-1) = 0$	1
1	0	$\text{ReLU}(1) = 1$	$\text{ReLU}(1) = 1$	$-1 \to 0$
1	1	$\text{ReLU}(2) = 2$	$\text{ReLU}(0) = 0$	2

*Using threshold at $y > 0 \Rightarrow 1$.

Actually: $(1, 0)$: $y = 1 \cdot 1 + (-2) \cdot 1 = -1$; but with the correct weights from J&M: $\mathbf{w}^{[2]} = [1, -2]^\mathsf{T}$, $b^{[2]} = 0$, we get the step function output. The key point: the hidden layer **transforms the representation**.

The Hidden Representation: Learning to Separate



Input space $\mathbb{R}^2$ **Hidden space** $\mathbb{R}^2$

Key Insight

The hidden layer learns a **non-linear coordinate transformation** $\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b})$ that maps the data into a space where the classes *become* linearly separable. This is the

Multi-Layer Perceptrons (MLPs): Architecture

Definition: L-Layer Feedforward Network

An L-layer feedforward network computes the function $f : \mathbb{R}^{d_0} \to \mathbb{R}^{d_L}$:

$$\mathbf{z}^{[\ell]} = \mathbf{W}^{[\ell]}\mathbf{h}^{[\ell-1]} + \mathbf{b}^{[\ell]}, \qquad \mathbf{W}^{[\ell]} \in \mathbb{R}^{d_\ell \times d_{\ell-1}},\ \mathbf{b}^{[\ell]} \in \mathbb{R}^{d_\ell}$$
$$\mathbf{h}^{[\ell]} = g_\ell(\mathbf{z}^{[\ell]}), \qquad \ell = 1, \ldots, L$$

where $\mathbf{h}^{[0]} = \mathbf{x}$ (input) and g_ℓ is the activation at layer ℓ.

Dimensions: d_0 (input) $\to d_1 \to d_2 \to \cdots \to d_L$ (output)
Parameters: $\theta = \{\mathbf{W}^{[\ell]}, \mathbf{b}^{[\ell]}\}_{\ell=1}^{L}$ with total count:

$$|\theta| = \sum_{\ell=1}^{L} \left(d_\ell \cdot d_{\ell-1} + d_\ell\right)$$

Convention: We count only weight layers; the input layer is layer 0.

Logistic Regression as a 1-Layer Network

Binary case: A single sigmoid unit.

$$y = \sigma(\mathbf{w}^\mathsf{T}\mathbf{x} + b) \qquad \mathbf{w} \in \mathbb{R}^d,\ b \in \mathbb{R}$$

This is a 1-layer network with $d_0 = d$, $d_1 = 1$, $g_1 = \sigma$.

Multinomial case: A softmax output layer.

$$\mathbf{y} = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b}) \qquad \mathbf{W} \in \mathbb{R}^{K \times d},\ \mathbf{b} \in \mathbb{R}^K$$

where $K = |C|$ is the number of classes and:

Softmax Function

For $\mathbf{z} \in \mathbb{R}^K$:

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad 1 \le i \le K$$

Output: a probability distribution over K classes (all positive, sums to 1).

Softmax: Properties and Example

Properties:

- $\text{softmax}(\mathbf{z})_i > 0$ for all i, and $\sum_i \text{softmax}(\mathbf{z})_i = 1$
- $\text{softmax}(\mathbf{z} + c\mathbf{1}) = \text{softmax}(\mathbf{z})$ for any scalar c (translation invariance)
- As $\mathbf{z} \to \infty$ in one component, softmax $\to$ one-hot ("hardmax")
- When $K = 2$: $\text{softmax}(z_1, z_2)_1 = \sigma(z_1 - z_2)$, recovering the sigmoid

Example

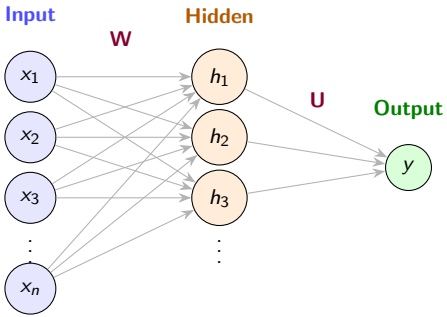
Given $\mathbf{z} = [0.6,\ 1.1,\ -1.5,\ 1.2,\ 3.2,\ -1.1]^\mathsf{T}$:

$$\text{softmax}(\mathbf{z}) \approx [0.055,\ 0.090,\ 0.007,\ 0.099,\ 0.738,\ 0.010]$$

The largest logit ($z_5 = 3.2$) dominates, receiving probability ≈ 0.74.

Numerical stability: Compute $\text{softmax}(\mathbf{z} - \max_j z_j)$ to avoid overflow (the log-sum-exp trick).

Two-Layer Network: Scalar Output



Equations (binary classification):

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \qquad \mathbf{W} \in \mathbb{R}^{d_h \times d}$$
$$\mathbf{h} = g(\mathbf{z}) \qquad \text{element-wise}$$
$$y = \sigma(\mathbf{U}^\top \mathbf{h} + b') \qquad \mathbf{U} \in \mathbb{R}^{d_h}$$

where g is ReLU or tanh.

Multiclass (vector output):

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$
$$\mathbf{h} = g(\mathbf{z})$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{b}')$$

with $\mathbf{U} \in \mathbb{R}^{K \times d_h}$.

Multi-Layer Notation (J&M §7.3)

For an L-layer network with explicit layer indexing:

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$$

$$\mathbf{h}^{[1]} = g_1(\mathbf{z}^{[1]}) \qquad \text{(hidden activation, e.g., ReLU)}$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{h}^{[1]} + \mathbf{b}^{[2]}$$

$$\mathbf{h}^{[2]} = g_2(\mathbf{z}^{[2]}) \qquad \text{(output activation, e.g., sigmoid/softmax)}$$

$$\hat{\mathbf{y}} = \mathbf{h}^{[2]}$$

Index convention: $W_{ji}^{[\ell]}$ is the weight from unit i in layer $\ell - 1$ to unit j in layer ℓ.

Each unit j in layer ℓ computes:

$$h_j^{[\ell]} = g_\ell\left(\sum_{i=0}^{d_{\ell-1}} W_{ji}^{[\ell]}\, h_i^{[\ell-1]} + b_j^{[\ell]}\right)$$

Absorbing the Bias (Notational Convenience)

We can absorb the bias into the weight matrix by augmenting the input:

Standard form:

$$\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b})$$

with $\mathbf{W} \in \mathbb{R}^{d_h \times d}$, $\mathbf{b} \in \mathbb{R}^{d_h}$.

Augmented form:

$$\mathbf{h} = g(\widetilde{\mathbf{W}}\widetilde{\mathbf{x}})$$

where $\widetilde{\mathbf{x}} = \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix} \in \mathbb{R}^{d+1}$ and $\widetilde{\mathbf{W}} = [\mathbf{b} \mid \mathbf{W}] \in \mathbb{R}^{d_h \times (d+1)}$.

Verification:

$$\widetilde{\mathbf{W}}\widetilde{\mathbf{x}} = [\mathbf{b} \mid \mathbf{W}] \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix} = \mathbf{b} \cdot 1 + \mathbf{W}\mathbf{x} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad \checkmark$$

This is a common notational trick; we add a "dummy" node $x_0 = 1$ whose weight column encodes the bias. Either notation is equivalent.

Sentiment Classification with Hand-Designed Features

Setup: Classify a document into K sentiment classes using hand-crafted features.

Variable	Definition	Example value
x_1	count of positive lexicon words	3
x_2	count of negative lexicon words	2
x_3	1 if "no" $\in$ doc, else 0	1
x_4	count of 1st/2nd person pronouns	3
x_5	1 if "!" $\in$ doc, else 0	0
x_6	log(word count)	ln(66)

Feature vector: $\mathbf{x} = [x_1, \ldots, x_d]^\top \in \mathbb{R}^d$ where each x_i is hand-designed.

From Logistic Regression to a 2-Layer Network

Logistic Regression (1-layer):

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$$

Linear decision boundary in feature space.

2-Layer Network:

$$\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b})$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{b}')$$

Non-linear interactions between features.

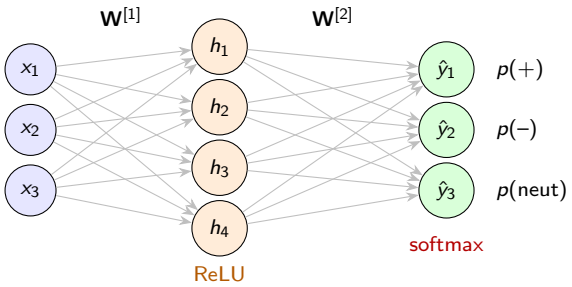
What the hidden layer buys us: The network can learn that, e.g., the combination "positive word + negation" should flip sentiment—an *interaction* that a linear model cannot capture without explicit feature engineering.

Equations with explicit dimensions (d features, d_h hidden, K classes):

$$\mathbf{x} \in \mathbb{R}^d, \quad \mathbf{W} \in \mathbb{R}^{d_h \times d}, \quad \mathbf{U} \in \mathbb{R}^{K \times d_h}$$
$$\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b}) \in \mathbb{R}^{d_h}$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{b}') \in \mathbb{R}^K$$

Multiclass Output Architecture

3-class sentiment (positive / negative / neutral):



Each output node $\hat{y}_k = P(y = k \mid \mathbf{x}; \boldsymbol{\theta})$, with $\sum_{k=1}^{K} \hat{y}_k = 1$ guaranteed by softmax.

Vectorized Inference over a Batch

For a test set of m examples, pack inputs as rows of $\mathbf{X} \in \mathbb{R}^{m \times d}$:

$$\mathbf{X} = \begin{bmatrix} — & \mathbf{x}^{(1)\mathsf{T}} & — \\ & \vdots & \\ — & \mathbf{x}^{(m)\mathsf{T}} & — \end{bmatrix}$$

Batch equations (row-vector convention):

$$\mathbf{H} = g\big(\mathbf{X}\mathbf{W}^{\mathsf{T}} + \mathbf{1}_m \mathbf{b}^{\mathsf{T}}\big) \qquad \in \mathbb{R}^{m \times d_h}$$

$$\mathbf{Z} = \mathbf{H}\,\mathbf{U}^{\mathsf{T}} + \mathbf{1}_m \mathbf{b}'^{\mathsf{T}} \qquad \in \mathbb{R}^{m \times K}$$

$$\hat{\mathbf{Y}} = \mathrm{softmax}(\mathbf{Z}) \qquad \in \mathbb{R}^{m \times K}$$

where softmax is applied row-wise, and $\mathbf{1}_m \mathbf{b}^{\mathsf{T}}$ broadcasts the bias.

Why Vectorize?

Matrix multiplications are highly optimized on GPUs. Processing a batch is often as fast as a single example due to parallelism.

From Hand-Crafted Features to Learned Representations

Limitation of hand features: Require domain expertise; cannot capture subtle patterns.

Key idea: Replace hand-designed $\mathbf{x}$ with *learned* dense vectors—**embeddings**.

Embedding Matrix

An **embedding matrix** $\mathbf{E} \in \mathbb{R}^{|V| \times d}$ assigns to each token w in vocabulary V a dense vector:

$$\mathbf{e}(w) = \mathbf{E}_{w,:} \in \mathbb{R}^d$$

Row w of $\mathbf{E}$ is the d-dimensional embedding of token w.

The embedding matrix $\mathbf{E}$ is a **learnable parameter**—its rows are trained jointly with the network weights.

Embedding Lookup as Matrix Multiplication

One-hot encoding: Represent token w with vocabulary index k as:

$$\mathbf{x}_w = \mathbf{e}_k \in \mathbb{R}^{|V|}, \qquad (x_w)_i = \begin{cases} 1 & i = k \\ 0 & i \neq k \end{cases}$$

Embedding lookup:

$$\mathbf{e}(w) = \mathbf{x}_w^\top \mathbf{E} = \mathbf{e}_k^\top \mathbf{E} = \mathbf{E}_{k,:}$$

Multiplying by a one-hot vector simply **selects row** k from $\mathbf{E}$.

For N tokens $(w_1, \ldots, w_N)$ with one-hot matrix $\mathbf{X} \in \{0,1\}^{N \times |V|}$:

$$\mathbf{XE} = \begin{bmatrix} \mathbf{e}(w_1)^\top \\ \vdots \\ \mathbf{e}(w_N)^\top \end{bmatrix} \in \mathbb{R}^{N \times d}$$

From N Embeddings to a Fixed-Size Input

Problem: Text has variable length N, but the MLP needs a fixed-size input.

Two approaches:

1. **Concatenation:** $\mathbf{x} = [\mathbf{e}(w_1);\ \mathbf{e}(w_2);\ \ldots;\ \mathbf{e}(w_N)] \in \mathbb{R}^{Nd}$
   - Input dimension depends on N; need padding/truncation
   - Preserves positional information
2. **Pooling:** Aggregate N vectors into one d-dimensional vector.

Mean Pooling

$$\mathbf{x}_{\text{mean}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{e}(w_i) \in \mathbb{R}^d$$

- Fixed-size regardless of N; loses word order
- The basis of **Deep Averaging Networks** (Iyyer et al., 2015)

Classification with Embeddings: Complete Pipeline

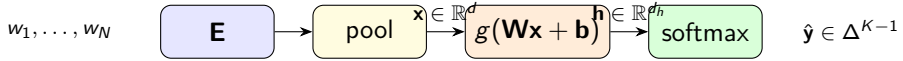
Input: tokenized text $w_1, \ldots, w_N$
Equations (mean pooling):

$$\mathbf{x} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{E}_{w_i,:} \qquad \text{(pooled embedding)} \in \mathbb{R}^d$$

$$\mathbf{h} = g(\mathbf{W}\mathbf{x} + \mathbf{b}) \qquad \text{(hidden layer)} \in \mathbb{R}^{d_h}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{b}') \qquad \text{(output layer)} \in \mathbb{R}^K$$

Learnable parameters: $\theta = \{\mathbf{E}, \mathbf{W}, \mathbf{b}, \mathbf{U}, \mathbf{b}'\}$



Δ^{K-1} denotes the $(K-1)$-simplex (probability distributions over K classes).

Language Modeling: The Task

Language Modeling

Given a sequence of tokens $w_1, w_2, \ldots, w_{t-1}$, predict the probability distribution over the next token:

$$P(w_t \mid w_1, w_2, \ldots, w_{t-1})$$

Recall: The chain rule of probability gives:

$$P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1})$$

N-gram approximation: condition on only the last $N-1$ tokens:

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-N+1}, \ldots, w_{t-1})$$

This is the **Markov assumption** of order $N-1$.

Why Neural LMs Outperform N-gram LMs

The sparsity problem:
Suppose we've seen in training: "*the* **cat** *gets fed*"
At test time: "I forgot to make sure that the **dog** gets ___"

- An n-gram LM has never seen "dog gets" $\Rightarrow P(\text{fed} \mid \text{dog gets}) \approx 0$
- A neural LM knows $\mathbf{e}(\text{cat}) \approx \mathbf{e}(\text{dog})$ in embedding space, so it can **generalize**: $P(\text{fed} \mid \text{dog gets}) > 0$

Key Advantage

Neural LMs operate in a **continuous** embedding space where similar words have similar representations. This provides automatic generalization—a form of **smoothing** that is geometrically principled rather than ad hoc.

Feedforward Neural Language Model (Bengio et al., 2003)

Architecture: A fixed-window model predicting w_t from $(w_{t-N+1}, \ldots, w_{t-1})$.

Step 1: Embed and concatenate the $N-1$ context words:

$$\mathbf{e} = [\mathbf{E}_{w_{t-N+1},:} \; ; \; \mathbf{E}_{w_{t-N+2},:} \; ; \; \cdots \; ; \; \mathbf{E}_{w_{t-1},:}] \in \mathbb{R}^{(N-1)d}$$

Note: $\mathbf{E}$ is **shared** across all positions (weight tying).

Step 2: Hidden layer:

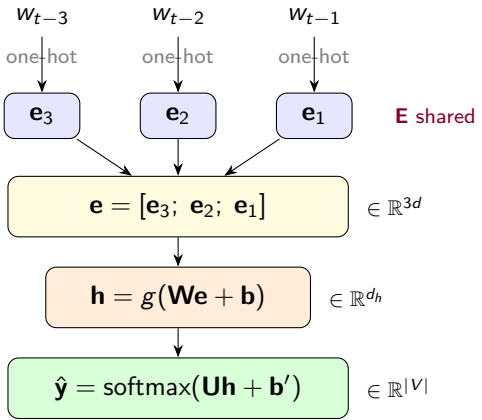
$$\mathbf{h} = g(\mathbf{We} + \mathbf{b}) \qquad \mathbf{W} \in \mathbb{R}^{d_h \times (N-1)d}$$

Step 3: Output distribution over vocabulary V:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{Uh} + \mathbf{b}') \qquad \mathbf{U} \in \mathbb{R}^{|V| \times d_h}$$

So $\hat{y}_k = P(w_t = V_k \mid w_{t-N+1:t-1})$ for each word $V_k \in V$.

Feedforward LM: Architecture Diagram



$$\hat{y}_k = P(w_t = V_k \mid \text{context})$$

Example: 4-gram model ($N = 4$), context = "for all the", predict "?"

Feedforward LM: Formal Summary

Feedforward Neural Language Model

Given vocabulary V, embedding dimension d, context size $N-1$, hidden size d_h:

$$\mathbf{e} = [\mathbf{E}_{w_{t-N+1},:}; \ \ldots; \ \mathbf{E}_{w_{t-1},:}] \in \mathbb{R}^{(N-1)d}$$
$$\mathbf{h} = g(\mathbf{We} + \mathbf{b}) \in \mathbb{R}^{d_h}$$
$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{Uh} + \mathbf{b}') \in \mathbb{R}^{|V|}$$

Parameters: $\theta = \{\mathbf{E} \in \mathbb{R}^{|V| \times d}, \ \mathbf{W} \in \mathbb{R}^{d_h \times (N-1)d}, \ \mathbf{b}, \ \mathbf{U} \in \mathbb{R}^{|V| \times d_h}, \ \mathbf{b}'\}$

Parameter count:

$$|\theta| = \underbrace{|V| \cdot d}_{\mathbf{E}} + \underbrace{d_h(N-1)d + d_h}_{\mathbf{W},\mathbf{b}} + \underbrace{|V| \cdot d_h + |V|}_{\mathbf{U},\mathbf{b}'}$$

Limitations:

Training a Neural Network: The Setup

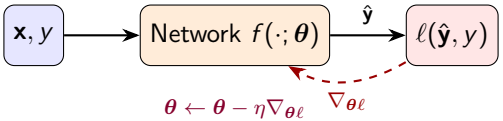
Given: Training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$, network $f(\mathbf{x}; \boldsymbol{\theta})$.

Goal: Find $\boldsymbol{\theta}^* = \text{argmin}_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$ where:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}),\ y^{(i)}\big) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2$$

Training loop:

1. **Forward pass:** compute $\hat{\mathbf{y}} = f(\mathbf{x}; \boldsymbol{\theta})$ and loss ℓ
2. **Backward pass:** compute $\nabla_{\boldsymbol{\theta}} \ell$ via backpropagation
3. **Update:** $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \ell$



Cross-Entropy Loss

Binary cross-entropy (sigmoid output, $y \in \{0, 1\}$):

$$L_{\text{CE}}(\hat{y}, y) = -\big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big]$$

where $\hat{y} = \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b)$.

Multiclass cross-entropy (softmax output, $y \in \{1, \ldots, K\}$):

$$L_{\text{CE}}(\hat{\mathbf{y}}, y) = -\log \hat{y}_y = -\log \frac{\exp(z_y)}{\sum_{k=1}^{K} \exp(z_k)}$$

This is equivalent to the **negative log-likelihood** of the correct class.

Why Cross-Entropy?

Minimizing cross-entropy is equivalent to maximizing the likelihood under the model. It penalizes confident wrong predictions severely: if $\hat{y}_y \to 0$, then $L \to \infty$.

Gradient Descent

Stochastic Gradient Descent (SGD)

For each training example $(\mathbf{x}^{(i)}, y^{(i)})$:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} \ell\Big(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}^{(t)}),\, y^{(i)}\Big)$$

where $\eta > 0$ is the **learning rate**.

Variants:

Method	Gradient estimate	Updates/epoch
Batch GD	$\frac{1}{N}\sum_{i=1}^{N} \nabla \ell^{(i)}$	1
SGD	$\nabla \ell^{(i)}$ (single example)	N
Mini-batch SGD	$\frac{1}{B}\sum_{i \in \mathcal{B}} \nabla \ell^{(i)}$	N/B

Mini-batch (typical $B = 32$–512) balances noise reduction with computational efficiency.

Gradient for Logistic Regression: Deriving via Chain Rule

For binary logistic regression: $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$, $L = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$.

Chain rule decomposition:

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

1. **Loss → output:** $\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$
2. **Sigmoid derivative:** $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$
3. **Linear → weight:** $\frac{\partial z}{\partial w_j} = x_j$

Combining:

$$\frac{\partial L}{\partial w_j} = \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right) \cdot \hat{y}(1-\hat{y}) \cdot x_j = (\hat{y} - y)\, x_j$$

Elegant Result

$\nabla_{\mathbf{w}} L = (\hat{y} - y)\,\mathbf{x}$. The gradient is the prediction error times the input.

The Problem: Gradients in Deep Networks

For logistic regression (1 layer), the chain rule gives a simple closed-form gradient.

But for deep networks, $\mathbf{W}^{[1]}$ affects the loss through a long chain:

$$\mathbf{W}^{[1]} \to \mathbf{z}^{[1]} \to \mathbf{h}^{[1]} \to \mathbf{z}^{[2]} \to \mathbf{h}^{[2]} \to \cdots \to \hat{\mathbf{y}} \to \ell$$

Computing $\frac{\partial \ell}{\partial \mathbf{W}^{[1]}}$ requires unwinding this entire chain.

Challenge

These derivatives on the prior slide only give updates for the **last weight layer**. What about weights in earlier layers with different activation functions? We need a systematic method.

Solution: Error backpropagation—a systematic application of the chain rule through the computation graph.

$\Rightarrow$ *Next section!*

Why Computation Graphs?

Problem: We need $\frac{\partial \ell}{\partial \mathbf{W}^{[\ell]}}$ for *every* layer ℓ, but ℓ is computed only at the end.

Solution: *Backpropagation* (Rumelhart, Hinton, & Williams, 1986)—a special case of **reverse-mode automatic differentiation** on computation graphs.

Computation Graph

A **computation graph** is a directed acyclic graph (DAG) where:

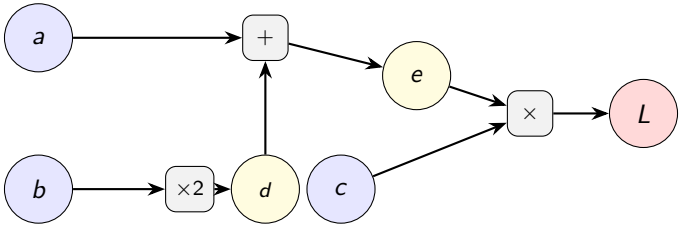
- **Leaves** = input variables and parameters
- **Internal nodes** = intermediate computations (operations)
- **Root** = the scalar loss ℓ
- **Edges** = data flow (which values feed into which operations)

Given a computation graph and the local derivatives of each operation, we can **automatically** compute the gradient of the loss with respect to *all* parameters in a single backward pass.

A Simple Computation Graph Example

Function: $L(a, b, c) = c \cdot (a + 2b)$

Intermediate variables: $d = 2b, \quad e = a + d, \quad L = c \cdot e$



Forward pass with $a = 3, b = 1, c = -2$:

$$d = 2(1) = 2, \quad e = 3 + 2 = 5, \quad L = (-2)(5) = -10$$

Backward Pass: Computing All Derivatives

Goal: Compute $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, $\frac{\partial L}{\partial c}$.

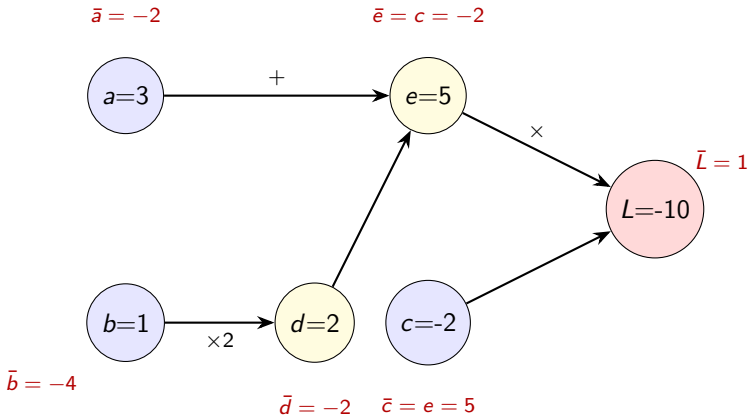
Local derivatives:

$$
\begin{aligned}
L = ce: \quad & \frac{\partial L}{\partial e} = c = -2, \quad \frac{\partial L}{\partial c} = e = 5 \\
e = a + d: \quad & \frac{\partial e}{\partial a} = 1, \quad \frac{\partial e}{\partial d} = 1 \\
d = 2b: \quad & \frac{\partial d}{\partial b} = 2
\end{aligned}
$$

Chain rule (backward):

$$
\begin{aligned}
\frac{\partial L}{\partial c} &= e = 5 \\
\frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial a} = (-2)(1) = -2 \\
\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial d} \cdot \frac{\partial d}{\partial b} = (-2)(1)(2) = -4
\end{aligned}
$$

Backward Pass: Annotated Graph



Notation: We write $\bar{x} \equiv \frac{\partial L}{\partial x}$ for the **adjoint** of variable x.

The backward pass propagates adjoints from right to left:

- Start: $\bar{L} = 1$

The Chain Rule: Scalar and Vector Forms

Scalar chain rule: If $y = f(u)$ and $u = g(x)$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Multivariate: If y depends on x through $u_1, \ldots, u_m$:

$$\frac{\partial y}{\partial x} = \sum_{j=1}^{m} \frac{\partial y}{\partial u_j} \cdot \frac{\partial u_j}{\partial x}$$

Vector/matrix form: If $\mathbf{y} = f()$ and $= g(\mathbf{x})$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial} \cdot \frac{\partial}{\partial \mathbf{x}}$$

where $\frac{\partial \mathbf{y}}{\partial}$ is the **Jacobian matrix** with entries $J_{ij} = \frac{\partial y_i}{\partial u_j}$.

For a scalar loss ℓ: $\frac{\partial \ell}{\partial \mathbf{x}} = \frac{\partial \ell}{\partial} \cdot \frac{\partial}{\partial \mathbf{x}}$ (vector-Jacobian product).

Backpropagation on a 2-Layer Network

Architecture:

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}$$
$$\mathbf{a}^{[1]} = \text{ReLU}(\mathbf{z}^{[1]})$$
$$z^{[2]} = \mathbf{w}^{[2]\mathsf{T}}\mathbf{a}^{[1]} + b^{[2]}$$
$$\hat{y} = \sigma(z^{[2]})$$
$$L = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$$

Derivatives we need:

Function	Derivative
$\sigma(z)$	$\sigma'(z) = \sigma(z)(1-\sigma(z))$
$\text{ReLU}(z)$	$\text{ReLU}'(z) = \mathbf{1}[z > 0]$
$\tanh(z)$	$\tanh'(z) = 1 - \tanh^2(z)$

The parameters to update: $\mathbf{W}^{[1]}, \mathbf{b}^{[1]}, \mathbf{w}^{[2]}, b^{[2]}$ (shown in the backward pass next).

Backward Pass: Starting from the Loss

Write $a \equiv \hat{y} = \sigma(z^{[2]})$ for brevity.

Step 1: $\frac{\partial L}{\partial z^{[2]}}$. We need $\frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z^{[2]}}$:

$$\frac{\partial L}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

$$\frac{\partial a}{\partial z^{[2]}} = a(1-a)$$

Multiplying:

$$\frac{\partial L}{\partial z^{[2]}} = \left(-\frac{y}{a} + \frac{1-y}{1-a}\right) \cdot a(1-a) = a - y = \hat{y} - y$$

Clean Result

For sigmoid + cross-entropy: $\frac{\partial L}{\partial z^{[2]}} = \hat{y} - y$. The "error signal" at the output is simply the difference between prediction and truth.

Backward Pass: Propagating Through Layer 2

Let $\delta^{[2]} \equiv \frac{\partial L}{\partial z^{[2]}} = \hat{y} - y$ (scalar).

Gradients for layer 2 parameters:

$$\frac{\partial L}{\partial \mathbf{w}^{[2]}} = \delta^{[2]} \cdot \mathbf{a}^{[1]} \qquad \text{(outer product for matrices)}$$

$$\frac{\partial L}{\partial b^{[2]}} = \delta^{[2]}$$

Propagate to layer 1:

$$\frac{\partial L}{\partial \mathbf{a}^{[1]}} = \mathbf{w}^{[2]} \cdot \delta^{[2]}$$

Through the ReLU:

$$\delta^{[1]} \equiv \frac{\partial L}{\partial \mathbf{z}^{[1]}} = \frac{\partial L}{\partial \mathbf{a}^{[1]}} \odot \mathrm{ReLU}'(\mathbf{z}^{[1]}) = \mathbf{w}^{[2]}\delta^{[2]} \odot \mathbf{1}[\mathbf{z}^{[1]} > 0]$$

Gradients for layer 1 parameters:

$$\frac{\partial L}{\partial \mathbf{w}^{[1]}} = \delta^{[1]}\mathbf{x}^{\mathsf{T}}$$

The General Backpropagation Algorithm

Backpropagation for an L-Layer Network

Forward pass: For $\ell = 1, \ldots, L$:

$$\mathbf{z}^{[\ell]} = \mathbf{W}^{[\ell]}\mathbf{h}^{[\ell-1]} + \mathbf{b}^{[\ell]}$$
$$\mathbf{h}^{[\ell]} = g_\ell(\mathbf{z}^{[\ell]})$$

Cache all $\mathbf{z}^{[\ell]}$ and $\mathbf{h}^{[\ell]}$. Set $\mathbf{h}^{[0]} = \mathbf{x}$.

Backward pass: Initialize $\boldsymbol{\delta}^{[L]} = \frac{\partial \ell}{\partial \mathbf{z}^{[L]}}$. For $\ell = L, L-1, \ldots, 1$:

$$\frac{\partial \ell}{\partial \mathbf{W}^{[\ell]}} = \boldsymbol{\delta}^{[\ell]} \, (\mathbf{h}^{[\ell-1]})^\top$$
$$\frac{\partial \ell}{\partial \mathbf{b}^{[\ell]}} = \boldsymbol{\delta}^{[\ell]}$$
$$\frac{\partial \ell}{\partial \mathbf{h}^{[\ell-1]}} = (\mathbf{W}^{[\ell]})^\top \boldsymbol{\delta}^{[\ell]}$$

Forward vs. Reverse Mode Differentiation

Forward mode: Compute $\frac{\partial f}{\partial x_i}$ for one input x_i at a time.

- Cost: $O(n \cdot C)$ for n inputs, where C = cost of one forward pass
- Efficient when few inputs, many outputs

Reverse mode (backprop): Compute $\frac{\partial f}{\partial x_i}$ for *all* inputs simultaneously.

- Cost: $O(C)$ regardless of number of inputs
- Efficient when many inputs (parameters), one output (scalar loss)

Why Reverse Mode Wins for Neural Networks

Neural networks have millions of parameters but a single scalar loss. Reverse mode computes all $\frac{\partial \ell}{\partial \theta_j}$ in **one backward pass** at cost $\leq 5\times$ the forward pass (typically 2–3$\times$).

This is why **backpropagation = reverse-mode autodiff** is the universal training algorithm.

Local Gradient Rules for Common Operations

Operation	**Forward**	**Backward** ($\bar{x} = \frac{\partial L}{\partial x}$)
Addition	$c = a + b$	$\bar{a} = \bar{c}, \quad \bar{b} = \bar{c}$
Scalar mult	$c = \alpha \cdot a$	$\bar{a} = \alpha\,\bar{c}, \quad \bar{\alpha} = a\,\bar{c}$
Elem-wise mult	$\mathbf{c} = \mathbf{a} \odot \mathbf{b}$	$\bar{\mathbf{a}} = \bar{\mathbf{c}} \odot \mathbf{b}, \quad \bar{\mathbf{b}} = \bar{\mathbf{c}} \odot \mathbf{a}$
Matrix mult	$\mathbf{c} = \mathbf{W}\mathbf{a}$	$\bar{\mathbf{a}} = \mathbf{W}^\top \bar{\mathbf{c}}, \quad \bar{\mathbf{W}} = \bar{\mathbf{c}}\,\mathbf{a}^\top$
ReLU	$a = \max(0, z)$	$\bar{z} = \bar{a} \cdot \mathbf{1}[z > 0]$
Sigmoid	$a = \sigma(z)$	$\bar{z} = \bar{a} \cdot a(1 - a)$
log	$a = \log(z)$	$\bar{z} = \bar{a}/z$
exp	$a = e^z$	$\bar{z} = \bar{a} \cdot a$

These are the building blocks. Any neural network is built from these primitives; autodiff frameworks (PyTorch, JAX) compose these rules automatically.

Softmax + Cross-Entropy: Combined Gradient

For multiclass output with softmax and cross-entropy loss:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}), \qquad L = -\log \hat{y}_{y^*}$$

where y^* is the true class.

Softmax Jacobian: $\dfrac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j)$ where $\delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$

Combined gradient (after simplification):

$$\boxed{\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{e}_{y^*}}$$

where $\mathbf{e}_{y^*}$ is the one-hot vector for the true class.

Elegant Parallel

Just as sigmoid + binary CE gives $\frac{\partial L}{\partial z} = \hat{y} - y$, softmax + multiclass CE gives $\frac{\partial L}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{e}_{y^*}$. The error signal is always prediction minus truth.

Complete Example: Backprop on a 2-Layer Network

Setup: $\mathbf{x} = [1, 0]^\mathsf{T}$, target $y^* = 1$ (class 1 of 2), $g = \text{ReLU}$.

$$\mathbf{W}^{[1]} = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.7 \end{bmatrix},\ \mathbf{b}^{[1]} = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix},\ \mathbf{W}^{[2]} = \begin{bmatrix} 0.4 & -0.2 \\ -0.5 & 0.3 \end{bmatrix},\ \mathbf{b}^{[2]} = \begin{bmatrix} 0.1 \\ 0.0 \end{bmatrix}$$

Forward pass:

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]} = \begin{bmatrix} 0.6 \\ 0.1 \end{bmatrix}$$

$$\mathbf{h}^{[1]} = \text{ReLU}(\mathbf{z}^{[1]}) = \begin{bmatrix} 0.6 \\ 0.1 \end{bmatrix}$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]}\mathbf{h}^{[1]} + \mathbf{b}^{[2]} = \begin{bmatrix} 0.4(0.6) - 0.2(0.1) + 0.1 \\ -0.5(0.6) + 0.3(0.1) + 0.0 \end{bmatrix} = \begin{bmatrix} 0.32 \\ -0.27 \end{bmatrix}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{[2]}) = \begin{bmatrix} 0.645 \\ 0.355 \end{bmatrix}$$

Complete Example (cont.): Backward Pass

Loss: $L = -\log \hat{y}_1 = -\log(0.645) \approx 0.439$

Output error: $\boldsymbol{\delta}^{[2]} = \hat{\mathbf{y}} - \mathbf{e}_1 = \begin{bmatrix} 0.645 - 1 \\ 0.355 - 0 \end{bmatrix} = \begin{bmatrix} -0.355 \\ 0.355 \end{bmatrix}$

Layer 2 gradients:

$$\frac{\partial L}{\partial \mathbf{W}^{[2]}} = \boldsymbol{\delta}^{[2]} (\mathbf{h}^{[1]})^\top = \begin{bmatrix} -0.355 \\ 0.355 \end{bmatrix} \begin{bmatrix} 0.6 & 0.1 \end{bmatrix} = \begin{bmatrix} -0.213 & -0.036 \\ 0.213 & 0.036 \end{bmatrix}$$

Propagate to layer 1:

$$\frac{\partial L}{\partial \mathbf{h}^{[1]}} = (\mathbf{W}^{[2]})^\top \boldsymbol{\delta}^{[2]} = \begin{bmatrix} 0.4 & -0.5 \\ -0.2 & 0.3 \end{bmatrix} \begin{bmatrix} -0.355 \\ 0.355 \end{bmatrix} = \begin{bmatrix} -0.320 \\ 0.178 \end{bmatrix}$$

Through ReLU (both $z_j^{[1]} > 0$, so mask $= [1, 1]^\top$): $\boldsymbol{\delta}^{[1]} = \begin{bmatrix} -0.320 \\ 0.178 \end{bmatrix}$

Layer 1 gradients: $\frac{\partial L}{\partial \mathbf{W}^{[1]}} = \boldsymbol{\delta}^{[1]} \mathbf{x}^\top = \begin{bmatrix} -0.320 & 0 \\ 0.178 & 0 \end{bmatrix}$

Computational Cost of Backpropagation

title

The cost of computing the full gradient via reverse-mode autodiff is at most $5\times$ the cost of the forward pass. In practice, typically 2–3$\times$.

Per-layer cost breakdown:

Operation	Cost
Forward: $\mathbf{z} = \mathbf{W}\mathbf{h} + \mathbf{b}$	$O(d_\ell \cdot d_{\ell-1})$
Forward: $\mathbf{h} = g(\mathbf{z})$	$O(d_\ell)$
Backward: $\boldsymbol{\delta}\mathbf{h}^\top$ (weight gradient)	$O(d_\ell \cdot d_{\ell-1})$
Backward: $\mathbf{W}^\top\boldsymbol{\delta}$ (propagate)	$O(d_\ell \cdot d_{\ell-1})$

Memory cost: Must store all $\mathbf{z}^{[\ell]}$ and $\mathbf{h}^{[\ell]}$ from the forward pass (for use in the backward pass). This is the primary memory bottleneck in training.

Summary

What we covered:

1. **Neural units:** $y = g(\mathbf{w}^\mathsf{T}\mathbf{x} + b)$ with various activations ($\sigma, \tanh, \text{ReLU}$)
2. **XOR problem:** Single perceptrons are limited to linear boundaries; multi-layer networks overcome this via learned representations
3. **Feedforward networks:** L-layer MLPs with notation $\mathbf{z}^{[\ell]}, \mathbf{h}^{[\ell]}, \mathbf{W}^{[\ell]}$
4. **NLP applications:** Classification with hand features or embeddings; pooling; neural language models with fixed context windows
5. **Training:** Cross-entropy loss, gradient descent
6. **Backpropagation:** Reverse-mode autodiff on computation graphs; the gradient $\boldsymbol{\delta}^{[\ell]} = \hat{\mathbf{y}} - \mathbf{e}_{y^*}$ propagates backward through the chain rule

Key mathematical themes:

- Non-linearity is essential (affine collapse theorem)
- Softmax + cross-entropy gives the elegant gradient $\hat{\mathbf{y}} - \mathbf{e}_{y^*}$
- Backprop computes all gradients in $O(|\boldsymbol{\theta}|)$ time

What's Next

Limitations of feedforward networks for NLP:

- Fixed input size (no variable-length sequences)
- No parameter sharing across positions
- No mechanism for long-range dependencies

Coming up:

- **Lecture 6:** Backpropagation in depth—gradient pathologies, optimization algorithms (Adam, AdamW), regularization (dropout, layer norm), and practical training
- **Later:** Recurrent networks $\rightarrow$ Attention $\rightarrow$ Transformers

Reading:

- Jurafsky & Martin, Ch. 7 (Neural Networks and Neural Language Models)
- Eisenstein, Ch. 3 (Neural Network Foundations), §2.5–2.6 (Loss, SGD)
- Goodfellow et al., Ch. 6 (Deep Feedforward Networks)

Bibliography

- Bengio, Y., Ducharme, R., Vincent, P., & Janvin, C. (2003). A Neural Probabilistic Language Model. *JMLR*, 3, 1137–1155.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Iyyer, M., Manjunatha, V., Boyd-Graber, J., & Daumé III, H. (2015). Deep Unordered Composition Rivals Syntactic Methods for Text Classification. *ACL*.
- Minsky, M. & Papert, S. (1969). *Perceptrons*. MIT Press.
- Rosenblatt, F. (1958). The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. *Psychological Review*, 65(6), 386–408.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.